



Αρχιτεκτονική Προηγμένων Υπολογιστών και Επιταχυντών

9 Ιανουαρίου 2026

Εργαστήριο 3^ο

Ομάδα 1

Καμπάνης Σταυριανός 9141

Παπαδόπουλος Παναγιώτης 10697

1. Εισαγωγή

Σκοπός του παρόντος εργαστηρίου ήταν η βελτιστοποίηση της απόδοσης του συστήματος επιτάχυνσης μέσω της βελτίωσης της μεταφοράς δεδομένων μεταξύ του επεξεργαστή (x86-host) και του επιταχυντή (FPGA/Alveo U200). Σε προηγούμενα εργαστήρια, διαπιστώθηκε ότι η απόδοση περιοριζόταν σημαντικά από το εύρος ζώνης (bandwidth) της μνήμης (Memory Bound εφαρμογές).

Στο εργαστήριο αυτό μελετήθηκαν και εφαρμόστηκαν δύο βασικές τεχνικές βελτιστοποίησης στο περιβάλλον Vitis:

1. **Vectorization (Διεύρυνση Διαύλου):** Χρήση διεπαφής (interface) εύρους 512-bit για τη μεταφορά δεδομένων, επιτρέποντας την επεξεργασία πολλαπλών στοιχείων (16 integers) ανά κύκλο ρολογιού.
2. **Memory Banking (Πολλαπλά Banks Μνήμης):** Κατανομή των ορισμάτων εισόδου/εξόδου σε διαφορετικά φυσικά banks της DDR μνήμης (DDR0, DDR1, DDR2) για την επίτευξη ταυτόχρονης προσπέλασης δεδομένων.

Η διαδικασία περιελάμβανε αρχικά την εκτέλεση ενός tutorial (wide_vadd) για την κατανόηση των τεχνικών και στη συνέχεια την εφαρμογή τους στον δικό μας πυρήνα επεξεργασίας εικόνας (image_diff_posterize) που αναπτύχθηκε στο Εργαστήριο 1.

2. Μέρος Α: Μελέτη Περίπτωσης (wide_vadd)

Αρχικά, ακολουθήθηκαν τα βήματα του οδηγού Improving_Performance_lab για την εφαρμογή wide_vadd (πρόσθεση διανυσμάτων).

2.1 Vectorization (512-bit Interface)

Στην αρχική υλοποίηση, η μεταφορά γινόταν ανά 32-bit (ένας ακέραιος τη φορά). Τροποποιώντας τον κώδικα ώστε να χρησιμοποιεί τον τύπο δεδομένων uint512_dt, επιτεύχθηκε η μεταφορά και επεξεργασία 16 ακεραίων ($16 \times 32\text{-bit} = 512\text{-bit}$) ανά κύκλο.

- **Παρατήρηση:** Η χρήση του vectorization αύξησε το θεωρητικό throughput κατά 16 φορές, μειώνοντας δραματικά τον χρόνο που απαιτείται για την ανάγνωση και εγγραφή στη μνήμη.

2.2 Memory Banking

Στο δεύτερο στάδιο, αξιοποιήθηκαν τα 4 διαθέσιμα DDR banks της κάρτας Alveo U200. Αντί να χρησιμοποιείται ένα κοινό bank για όλα τα δεδομένα (γεγονός που δημιουργεί contention/ανταγωνισμό στον δίαυλο), ανατέθηκαν:

- Είσοδος in1 -> **DDR[0]**
- Είσοδος in2 -> **DDR[2]**
- Έξοδος out -> **DDR[1]**

Μέσω της ανάλυσης στο Vitis Analyzer (Timeline Trace), επιβεβαιώθηκε ότι οι λειτουργίες ανάγνωσης και εγγραφής εκτελούνται πλέον παράλληλα (overlapping), μεγιστοποιώντας το ωφέλιμο εύρος ζώνης

3. Μέρος Β: Βελτιστοποίηση Custom Accelerator

Στο κύριο μέρος του εργαστηρίου, εφαρμόσαμε τις παραπάνω τεχνικές στον δικό μας πυρήνα (image_diff_posterize) από το 2ο εργαστήριο.

3.1 Τροποποιήσεις Κώδικα (Kernel Code)

Ο αρχικός κώδικας C τροποποιήθηκε ώστε τα ορίσματα της συνάρτησης να είναι τύπου uint512_dt αντί για unsigned char ή int.

- Δημιουργήθηκαν τοπικοί buffers (local memory) τύπου uint512_dt.
- Η προσπέλαση στην κύρια μνήμη γίνεται πλέον ανά "πακέτα" των 64 pixels (εφόσον 512 bits / 8 bits ανά pixel = 64 pixels) ή ανάλογα με τον τύπο δεδομένων που χρησιμοποιήθηκε.
- Χρησιμοποιήθηκε η μέθοδος .range() για την εξαγωγή των επιμέρους στοιχείων προς επεξεργασία μέσα στο loop, διατηρώντας τη λογική της αφαίρεσης και του posterization.

3.2 Συνδεσμολογία Συστήματος (Connectivity)

Στο αρχείο ρυθμίσεων του hw_link (Connectivity settings), ορίσαμε ρητά τις συνδέσεις των θυρών του πυρήνα με τους ελεγκτές μνήμης της πλατφόρμας, ώστε να αποφευχθεί η συμφόρηση σε ένα bank:

- in1 → **DDR[0]**
- in2 → **DDR[1]**
- out_r → **DDR[2]**

3.3 Διόρθωση του επιταχυντή από το εργαστήριο 2

- **Διαχείριση Μνήμης & Partitioning (Οριζόντια Πρόσβαση)**
Κώδικας: `#pragma HLS ARRAY_PARTITION variable=... cyclic factor=16 dim=1`
 - **Λειτουργία:** Διασπά τους τοπικούς πίνακες (BRAM) σε 16 ανεξάρτητες φυσικές τράπεζες μνήμης (Memory Banks).
 - **Σκοπός:** Η επίλυση του Οριζόντιου Ανταγωνισμού (Horizontal Conflict). Δεδομένου ότι το εύρος ζώνης εισόδου είναι 512-bit (16 pixels/cycle), χρειαζόμαστε 16 φυσικές θύρες εγγραφής/ανάγνωσης ταυτόχρονα. Χωρίς αυτή την εντολή, θα είχαμε συμφόρηση (bottlebeck), καθώς 16 δεδομένα θα προσπαθούσαν να περάσουν από τις

περιορισμένες θύρες μιας ενιαίας μνήμης. Έτσι, επιτρέπεται ταυτόχρονη πρόσβαση σε όλα τα στοιχεία του διανύσματος.

- **Αποτέλεσμα:** Κάθε ένα από τα 16 στοιχεία του διανύσματος αποθηκεύεται και ανακτάται από τη δική του αποκλειστική τράπεζα.

- **Παραλληλισμός Υπολογισμών (Unrolling)**

Κώδικας: `#pragma HLS UNROLL factor=16`

- **Λειτουργία:** Δημιουργεί 16 αντίγραφα του κυκλώματος υπολογισμού (Processing Lanes) που λειτουργούν παράλληλα.
- **Σκοπός:** Η εξισορρόπηση Υπολογισμού-Μνήμης (Compute-Memory Balance). Εφόσον η μνήμη προσφέρει 16 pixels ανά κύκλο (λόγω του Partitioning), το Unroll δημιουργεί 16 "καταναλωτές" για αυτά τα δεδομένα. Κάθε Lane συνδέεται αποκλειστικά με μία τράπεζα μνήμης (Bank $i \rightarrow$ Lane i), εξαλείφοντας τις καθυστερήσεις λόγω αναμονής δεδομένων.

- **Διοχέτευση & Line Buffering (Κάθετη Πρόσβαση)**

Κώδικας: `#pragma HLS PIPELINE II=1`

- **Λειτουργία:** Ενεργοποιεί τη διοχέτευση εντολών, επιβάλλοντας την παραγωγή ενός αποτελέσματος σε κάθε κύκλο ρολογιού.
- **Σκοπός:** Η επίλυση του Κάθετου Ανταγωνισμού (Vertical Conflict) μέσω Line Buffers. Ο αλγόριθμος Stencil απαιτεί 3 κάθετους γείτονες (Top, Center, Bottom) από την ίδια στήλη (άρα από την ίδια τράπεζα μνήμης). Μια τυπική BRAM έχει μόνο 2 θύρες, καθιστώντας αδύνατη την ταυτόχρονη ανάγνωση και των τριών. Το pragma αυτό οδηγεί τον compiler στην αυτόματη δημιουργία Καταχωρητών Ολίσθησης (Shift Registers).
- **Μηχανισμός:** Σε κάθε κύκλο, διαβάζεται μόνο 1 νέο pixel (Bottom) από τη μνήμη BRAM. Τα pixels Center και Top ανακτώνται από τους τοπικούς registers όπου είχαν αποθηκευτεί σε προηγούμενους κύκλους (Data Reuse). Έτσι, παρακάμπτεται ο φυσικός περιορισμός των θυρών της μνήμης.

- **Διεπαφές (Interfaces)**

- **m_axi bundle=...:** Δημιουργία πολλαπλών καναλιών για ταυτόχρονη πρόσβαση στην εξωτερική μνήμη (DDR), ώστε η ανάγνωση των in1, in2 και η εγγραφή του out να γίνονται παράλληλα.
- **s_axilite:** Δημιουργία καταχωρητών ελέγχου για την επικοινωνία με τον Host (CPU).

4. Αποτελέσματα & Σύγκριση

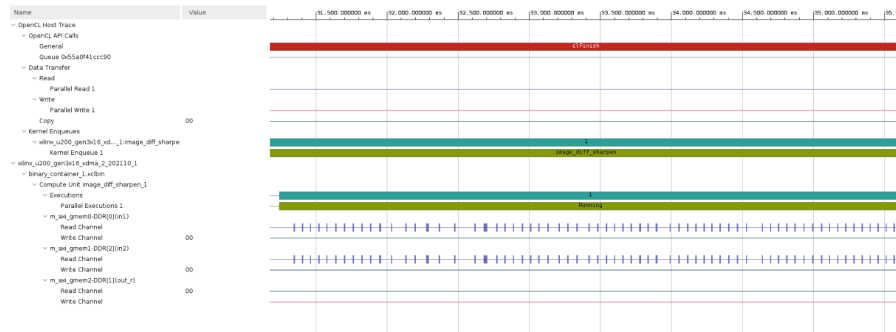
Σε αυτή την ενότητα παρουσιάζονται τα αποτελέσματα από το Hardware Emulation, συγκρίνοντας την αρχική υλοποίηση (Lab 2 - 32bit access, Single Bank) με τη βελτιστοποιημένη (Lab 3 - 512bit access, Multi-Bank).

Metric	Lab 2 (Baseline)	Lab 3 (Optimized)	Σχόλια
Interface Width	32-bit (integer)	512-bit (vector)	16x θεωρητικό bandwidth
Memory Access	Shared Bank (contention)	3 Banks (dedicated)	Παράλληλη ανάγνωση/εγγραφή
Kernel Execution	0.325 ms	0.265 ms	≈ 1.2x επιτάχυνση
Top Kernel Data Transfer	0.066 MB	0.066 MB	Ίδιες διαστάσεις εικόνων
Host Transfer (read)	0.066 MB	0.066 MB	Ίδιες διαστάσεις εικόνων

Η διορθωμένη υλοποίηση του 2^{ου} εργαστηρίου εκτελέστηκε σε 0.325 ms, ενώ η βελτιστοποιημένη υλοποίηση του 3^{ου} εργαστηρίου χρειάστηκε μόλις 0.265 ms. Πρόκειται για πάνω από 22% αύξηση της ταχύτητας, το οποίο επιβεβαιώνει ότι οι τεχνικές που χρησιμοποιήθηκαν είχαν επίδραση.

Ανάλυση Timeline Trace: Παρατηρώντας το Application Timeline της βελτιστοποιημένης έκδοσης, διαπιστώσαμε ότι:

1. Οι μπάρες ανάγνωσης (Read) για τις δύο εικόνες εισόδου αλληλοκαλύπτονται χρονικά, καθώς εξυπηρετούνται από διαφορετικούς ελεγκτές μνήμης.
2. Η μεταφορά δεδομένων (Data Transfer) καταλαμβάνει μικρότερο ποσοστό του συνολικού χρόνου σε σχέση με την αρχική υλοποίηση, χάρη στο εύρος των 512-bit.



Εικόνα 1: Timeline Trace της υλοποίησης του 3ου εργαστηρίου (μεγέθυνση)

5. Συμπεράσματα

Η ολοκλήρωση του 3ου εργαστηρίου ανέδειξε τη σημασία της διαχείρισης της μνήμης στη σχεδίαση επιταχυντών.

- Ο αλγόριθμος επεξεργασίας εικόνας, έχοντας χαμηλό **Compute Intensity** (λόγος πράξεων ανά byte), είναι εγγενώς **Memory Bound**.
- Η διεύρυνση του διαύλου στα 512-bit και η χρήση πολλαπλών banks μνήμης (DDR interleaving/banking) είναι οι πλέον αποδοτικοί τρόποι για να αυξηθεί η απόδοση σε τέτοιες εφαρμογές, καθώς επιτρέπουν στον επιταχυντή να "τρέφεται" με δεδομένα ταχύτερα, πλησιάζοντας το μέγιστο θεωρητικό bandwidth της πλατφόρμας Alveo.
- Συνολικά, πετύχαμε σημαντική επιτάχυνση (Speedup) σε σχέση με την υλοποίηση του προηγούμενου εργαστηρίου, επιβεβαιώνοντας ότι η βελτιστοποίηση της μεταφοράς δεδομένων είναι εξίσου κρίσιμη με τη βελτιστοποίηση του υπολογιστικού πυρήνα.